Evaluation of a set of functions related to Ripley's K analysis

Gabriel Arellano

2026-05-20

Goal

Evaluate, understand, and decide over a set of related functions that count individuals in circular areas and compare those counts with a null expectation, implementing Ripley's K test. At the moment, this is an isolated set of functions; see "1-dependencies-and-priorities-2026-05-20.pdf" report.

This set contains the following functions:

- "CalcRingArea" in spatial > RipUvK.r
- "partialcirclearea" in spatial > RipUvK.r
- "circlearea" in spatial > RipUvK.r
- "Annuli" in spatial > RipUvK.r
- "NDcount" in spatial > NeighborDensityFun.r
- "NeighborDensities" in spatial > NeighborDensityFun.r
- "RipUvK" in spatial > RipUvK.r

Overall criteria

These functions are located in two different scripts but all are related to the same type of calculation: counting number of neighbors in a given area, and compare that with some null expectation (Ripley's K function). The set covers all code in RipUvK.r and NeighborDensityFun.r original scripts.

This type of calculation is widely used in ForestGEO / ATFS, for example to estimate competition around target individuals and so on. Perhaps not to much for point pattern analyses. But,

in general, counting points in certain areas around target individuals (or locations) is valuable. Perhaps most of these functions can be incorporated into a more general function that counts (or summarizes) data belonging to different classes, within areas that have a minimum and a maximum radius.

Modelling or comparison with null expectations is less valuable for the fgeo2 package. The Ripley's K function is surely maintained by other packages. Although it's not a high-maintenance test, it is a clear candidate to remove, and substitute by a more general tutorial on how to apply point pattern analysis to forest data.

Preliminaries for code evaluation

First, we declare the path to fgeo2 folder and all the functionality needed, including CTFS / fgeo functions, external packages, and auxiliary code:

```
# This is my local path. You may have to adapt this.
g <- regexpr("fgeo2", getwd())
path <- substr(getwd(), 1, g + attr(g, "match.length"))

# Source useful functionality:
source(paste0(path, "evaluation/0-code-to-source.r"))

# Other functionality:
library(igraph)
```

```
Warning: package 'igraph' was built under R version 4.5.2
```

```
Attaching package: 'igraph'
```

```
The following objects are masked from 'package:stats':

    decompose, spectrum
```

```
The following object is masked from 'package:base':

    union
```

```
library(fgeo)
```

```
-- Attaching packages ---------------------------------------- fgeo 1.1.4 --

v fgeo.analyze 1.1.15     v fgeo.tool    1.2.10
v fgeo.plot    1.1.11     v fgeo.x       1.1.4

-- Conflicts ------------------------------------------------ fgeo_conflicts() --
x fgeo.tool::filter() masks stats::filter()
```

```
# This chunk of code will source all the scripts:
f1 <- list.files(paste0(path, "01_utilities/R/new/"), full.names = TRUE)
f2 <- list.files(paste0(path, "02_biomass/R/new/"), full.names = TRUE)
f3 <- list.files(paste0(path, "03_spatial/R/new/"), full.names = TRUE)
f4 <- list.files(paste0(path, "04_abundance_diversity/R/new/"), full.names = TRUE)
f5 <- list.files(paste0(path, "05_demography_growth/R/new/"), full.names = TRUE)
f6 <- list.files(paste0(path, "06_topography_habitat/R/new/"), full.names = TRUE)
f7 <- list.files(paste0(path, "07_mapping_plotting/R/new/"), full.names = TRUE)
ff <- c(f1, f2, f3, f4, f5, f6, f7)
for(f in ff) source(f)
```

Evaluation of specific functions

```
# Use this to see the source code, manually.

CalcRingArea
```

```
function (data, radius, plotdim)
{
    nopts <- dim(data)[1]
    internalArea <- numeric()
    for (i in 1:nopts) {
        xdist <- data$gx[i]
        if (plotdim[1] - data$gx[i] < xdist)
            xdist <- plotdim[1] - data$gx[i]
        internalArea[i] <- partialcirclearea(radius, xdist, data$gy[i],
            plotdim[2] - data$gy[i])
    }
    return(list(total = sum(internalArea), each = internalArea))
}
```

```
partialcirclearea
```

```
function (r, c2, cy1, cy2)
{
    if (cy1 <= cy2) {
        c1 <- cy1
        c3 <- cy2
    }
    else {
        c1 <- cy2
        c3 <- cy1
    }
    if (r > c1) {
        alpha1 <- acos(c1/r)
        y1 <- sqrt(r * r - c1 * c1)
    }
    if (r > c2) {
        alpha2 <- acos(c2/r)
        y2 <- sqrt(r * r - c2 * c2)
    }
    if (r > c3) {
        alpha3 <- acos(c3/r)
        y3 <- sqrt(r * r - c3 * c3)
    }
    cornerdist1 <- sqrt(c1 * c1 + c2 * c2)
    cornerdist3 <- sqrt(c2 * c2 + c3 * c3)
    if (r <= c1 && r <= c2)
        area <- circlearea(r)
    else if (r > c1 && r <= c2 && r <= c3)
        area <- r * r * (pi - alpha1) + c1 * y1
    else if (r <= c1 && r > c2 && r <= c3)
        area <- r * r * (pi - alpha2) + c2 * y2
    else if (r > c1 && r > c2 && r <= c3) {
        if (r > cornerdist1)
            area <- r * r * (3 * pi/4 - alpha1/2 - alpha2/2) +
                c1 * c2 + (c1 * y1 + c2 * y2)/2
        else area <- r * r * (pi - alpha1 - alpha2) + c1 * y1 +
            c2 * y2
    }
    else if (r <= c2 && r > c1 && r > c3)
        area <- r * r * (pi - alpha1 - alpha3) + c1 * y1 + c3 *
            y3
```

```
    else if (r > c2 && r > c1 && r > c3) {
        if (r > cornerdist3)
            area <- r * r * (pi - alpha1 - alpha3)/2 + c1 * c2 +
                c2 * c3 + (c1 * y1 + c3 * y3)/2
        else if (r > cornerdist1 && r <= cornerdist3)
            area <- r * r * (3 * pi/4 - alpha1/2 - alpha2/2 -
                alpha3) + (c1 * y1 + c2 * y2)/2 + c3 * y3 + c1 *
                c2
        else if (r <= cornerdist1 && r <= cornerdist3)
            area <- r * r * (pi - alpha1 - alpha2 - alpha3) +
                c1 * y1 + c2 * y2 + c3 * y3
    }
    return(area)
}
```

`circlearea`

```
function (r)
{
    return(pi * r^2)
}
```

`Annuli`

```
function (spdata, r, plotdim)
{
    TotalAreaPerRing <- numeric()
    AreaPerCircle <- matrix(nrow = dim(spdata)[1], ncol = length(r) +
        1)
    TotalAreaPerRing[1] <- 0
    AreaPerCircle[, 1] <- 0
    for (i in 1:length(r)) {
        ringarea <- CalcRingArea(spdata, r[i], plotdim)
        TotalAreaPerRing[i + 1] <- ringarea$total
        AreaPerCircle[, i + 1] <- ringarea$each
    }
    AreaPerRing <- t(apply(AreaPerCircle, 1, diff))
    return(list(total = diff(TotalAreaPerRing), each = AreaPerRing))
}
```

NDcount

```
function (censdata, r = 20, plotdim = c(1000, 500))
{
    sppi.pts <- with(censdata, data.frame(gx, gy))
    areas <- CalcRingArea(sppi.pts, r, plotdim)$each
    poly <- spoints(c(0, 0, plotdim[1], 0, plotdim[1], plotdim[2],
        0, plotdim[2]))
    counts <- khat(sppi.pts, poly = poly, r, newstyle = TRUE)$counts
    output <- counts * pi * r^2/areas
    return(as.vector(output))
}
```

NeighborDensities

```
function (censdata, censdata2 = NULL, r = 20, plotdim = c(1000,
    500), mindbh = 10, type = "count", include = c("A"))
{
    ptm <- proc.time()
    if (is.null(censdata2))
        censdata2 <- censdata
    n <- nrow(censdata2)
    output <- matrix(NA, ncol = 2, nrow = n)
    dimnames(output)[[2]] <- c("con.sp", "het.sp")
    if (type == "count") {
        spd <- subset(censdata, !is.na(gx) & !is.na(gy) & !duplicated(tag) &
            dbh >= mindbh & status %in% include)
        for (i in 1:n) {
            focal <- censdata2[i, ]
            if (is.na(focal$gx) | is.na(focal$gy) | duplicated(focal$tag))
                output[i, 1:2] <- NA
            else {
                poly <- with(focal, spoints(c(gx - r, gy - r,
                  gx - r, gy + r, gx + r, gy + r, gx + r, gy -
                    r)))
                use <- inpip(spoints(rbind(spd$gx, spd$gy)),
                  poly)
                if (length(use) == 0)
                  output[i, 1:2] <- 0
                else {
                  incircle <- sqrt(dsquare(spoints(rbind(spd$gx[use],
```

```
                spd$gy[use])), spoints(rbind(focal$gx, focal$gy)))) <=
                r
              nn <- use[incircle]
              consp <- spd$sp[nn] == focal$sp
              area <- CalcRingArea(data.frame(gx = focal$gx,
                gy = focal$gy), r, plotdim)$each
              output[i, 1] <- length(nn[consp]) * (pi * r^2/area)
              output[i, 2] <- length(nn[!consp]) * (pi *
                r^2/area)
            }
        }
        if (i %in% seq(5000, n + 5000, 5000))
            cat(i, "of", n, " elapsed time = ", (proc.time() -
              ptm)[3], "seconds", "\n")
    }
}
if (type == "basal") {
    spd <- subset(censdata, !is.na(gx) & !is.na(gy) & !is.na(dbh) &
        dbh >= mindbh & status %in% include)
    for (i in 1:n) {
        focal <- censdata2[i, ]
        if (is.na(focal$gx) | is.na(focal$gy))
            output[i, 1:2] <- NA
        else {
            poly <- with(focal, spoints(c(gx - r, gy - r,
              gx - r, gy + r, gx + r, gy + r, gx + r, gy -
                r)))
            use <- inpip(spoints(rbind(spd$gx, spd$gy)),
              poly)
            if (length(use) == 0)
              output[i, 1:2] <- 0
            else {
              incircle <- sqrt(dsquare(spoints(rbind(spd$gx[use],
                spd$gy[use])), spoints(rbind(focal$gx, focal$gy)))) <=
                r
              nn <- use[incircle]
              area <- CalcRingArea(data.frame(gx = focal$gx,
                gy = focal$gy), r, plotdim)$each
              BA <- circlearea(spd$dbh[nn]/20)
              consp <- spd$sp[nn] == focal$sp
              output[i, 1] <- sum(BA[consp], na.rm = TRUE) *
                (pi * r^2/area)
              output[i, 2] <- sum(BA[!consp], na.rm = TRUE) *
```

```
                    (pi * r^2/area)
                }
            }
            if (i %in% seq(5000, n + 5000, 5000))
                cat(i, "of", n, " elapsed time = ", (proc.time() -
                  ptm)[3], "seconds", "\n")
        }
    }
    cat("Total elapsed time = ", (proc.time() - ptm)[3], "seconds",
        "\n")
    return(output)
}
```

RipUvK

```
function (splitdata, plotdim = c(1000, 500), rseq = c(5, 10,
    20, 30, 40, 50), mindbh = 10, xcol = "gx", ycol = "gy", debug = FALSE,
    show = FALSE)
{
    spp.names <- names(splitdata)
    plotarea <- plotdim[1] * plotdim[2]/10000
    omega <- matrix(nrow = length(spp.names), ncol = length(rseq))
    rip.list <- Ovalue <- list()
    abund <- numeric()
    poly <- spoints(c(0, 0, plotdim[1], 0, plotdim[1], plotdim[2],
        0, plotdim[2]))
    sppcounter <- 1
    for (i in 1:length(spp.names)) {
        if (mindbh == 0)
            size <- rep(0, dim(splitdata[[i]])[1])
        else size <- splitdata[[i]]$dbh
        if (is.null(splitdata[[i]]$status))
            alivestatus <- rep("A", dim(splitdata[[i]])[1])
        else alivestatus <- splitdata[[i]]$status
        gx <- (splitdata[[i]][, xcol])
        gy <- (splitdata[[i]][, ycol])
        include <- which(alivestatus == "A" & size >= mindbh &
            gx >= 0 & gx < plotdim[1] & gy >= 0 & gy < plotdim[2])
        inc <- (alivestatus == "A" & size >= mindbh & gx >= 0 &
            gx < plotdim[1] & gy >= 0 & gy < plotdim[2])
        gx <- gx[include]
        gy <- gy[include]
```

```
        abund[i] <- length(gx)
        if (debug)
            browser()
        if (abund[i] > 1) {
            sppi.pts <- spoints(rbind(gx, gy), length(gx))
            rip.list[[i]] <- khat(as.points(sppi.pts), poly = poly,
                rseq, newstyle = TRUE)
            ringarea <- Annuli(data.frame(gx, gy), rseq, plotdim)
           Ovalue[[i]] <- colSums(rip.list[[i]]$counts)/(ringarea$total/10000)
            rip.list[[i]]$area <- ringarea$each
            lambda <- abund[i]/plotarea
            if (debug)
                browser()
            omega[i, ] <- Ovalue[[i]]/lambda
            if (show) {
                if (sppcounter == 1)
                  plot(i, omega[i, 1], log = "y", xlim = c(0,
                    1200), ylim = c(0.1, 60))
                else points(i, omega[i, 1])
            }
            sppcounter <- sppcounter + 1
        }
        else rip.list[[i]] <- Ovalue[[i]] <- NA
    }
   names(Ovalue) <- names(rip.list) <- names(abund) <- rownames(omega) <- spp.names
    nopts <- length(rseq)
    midpts <- c(0, rseq[-nopts]) + diff(c(0, rseq))/2
    return(list(K = rip.list, O = Ovalue, omega = data.frame(omega),
        abund = abund, midpts = midpts))
}
```

The function circlearea is trivial; while the partialcirclearea makes edge corrections considering the dimensions of the plot.

CalcRingArea seems to be just the accumulated sum of (possibly truncated) circles, one for each point in a dataset. Note:

- it calculates areas of circles, not rings, despite its name
- it does not correct for overlap, so it counts many times the same areas

RipUvK is the highest-level function in this group of functions. It runs over each species calculating a series of spatial statistics for con-specifics at different distances. It seems easy to use, it is not very long. It is not very flexible, and calls external functions of undefined packages: khat, spoints.